

Empirical Evaluation of Monte Carlo Sampling Strategies for Numerical Integration: Accuracy, Convergence, and Computational Performance

Abstract

Monte Carlo methods are a class of stochastic algorithms [1], [6] that estimate numerical quantities through repeated random sampling and are widely used in scientific computing, computational physics, quantitative finance, and machine learning. This study presents an empirical evaluation of Monte Carlo sampling strategies for numerical integration through the estimation of the mathematical constant π using a reproducible C++ implementation. Standard

Monte Carlo integration was evaluated across sample sizes ranging from 10^3 to 10^7 , with estimation accuracy, convergence behavior, computational runtime, estimator variance, and confidence intervals analyzed quantitatively. The study further compares Standard Monte Carlo with Stratified Monte Carlo to investigate the impact of variance reduction on numerical accuracy and computational performance. Experimental results demonstrate convergence consistent with the Law of Large Numbers [1], while stratified sampling consistently achieved lower mean absolute error than Standard Monte Carlo across all evaluated sample sizes without requiring a substantial increase in computational cost. Although deterministic numerical integration methods remain more accurate for low-dimensional problems [4], the findings demonstrate the importance of sampling strategy in improving estimator efficiency and provide a reproducible experimental framework that can be extended to advanced variance reduction techniques, high-dimensional numerical integration, and parallel scientific computing.

1. Introduction

Monte Carlo methods are among the most widely used computational techniques [1], [5], [6] for solving numerical problems that are difficult or impossible to solve analytically. The historical development of Monte Carlo methods has been documented by one of its pioneers, Stanislaw Ulam [2]. Rather than relying on deterministic calculations, Monte Carlo algorithms estimate quantities through repeated random sampling. Their simplicity, flexibility, and scalability [5] have made them fundamental tools in scientific computing, computational physics, quantitative finance, machine learning, computer graphics, robotics, and Bayesian statistics.

One of the most widely used benchmark problems for introducing Monte Carlo integration is the numerical estimation of the mathematical constant π . Although estimating π is not itself a difficult computational problem, it provides an intuitive and reproducible framework for evaluating estimator accuracy, convergence behavior, and computational performance because the exact analytical value is known. This example illustrates the broader principle underlying Monte Carlo integration: accurate numerical estimates can emerge from sufficiently large collections of random samples.

The objective of this project is to investigate the performance of Monte Carlo integration by implementing a π estimator in C++. The implementation generates uniformly distributed random points within a unit square and estimates π by computing the proportion of points that fall inside an inscribed quarter circle. Experimental results are analyzed across multiple sample sizes to examine convergence behavior and estimation accuracy.

Beyond implementing the algorithm, this study compares Monte Carlo integration with classical deterministic numerical integration techniques, including the Midpoint Rule and Simpson's Rule. The comparison highlights the trade-offs between deterministic and stochastic approaches with respect to computational efficiency, convergence characteristics, scalability, and practical applications.

Although estimating π is a classical educational example, the underlying methodology extends to significantly more complex problems involving high-dimensional integration, uncertainty quantification, optimization, and probabilistic modeling. Consequently, this project serves not only as an introduction to Monte Carlo simulation but also as a foundation for understanding computational techniques that play an important role in modern computer science research.

Contributions of This Study

This study makes three primary contributions. First, it develops a reproducible C++ framework for evaluating Monte Carlo integration across multiple sample sizes using automated benchmarking and modern pseudo-random number generation. Second, it empirically analyzes estimator accuracy, convergence behavior, computational runtime, estimator variance, and

confidence intervals through systematic experimentation. Third, it compares Standard Monte Carlo and Stratified Monte Carlo to evaluate the impact of variance reduction on numerical accuracy and computational performance, providing a reproducible foundation for future investigations of more advanced Monte Carlo sampling methods.

Research Objectives

This study is designed to investigate the effectiveness of Monte Carlo integration for numerical approximation through the following research questions:

- **RQ1:** How does the estimation accuracy of Monte Carlo integration change as the number of randomly generated samples increases?
- **RQ2:** How does the convergence behavior of Monte Carlo integration compare with classical deterministic numerical integration methods such as the Midpoint Rule and Simpson's Rule?
- **RQ3:** What trade-offs exist between computational performance and estimation accuracy as the sample size increases?

Based on probability theory and the Law of Large Numbers, the following hypotheses are proposed:

- **H1:** Increasing the number of random samples will reduce the estimation error of the Monte Carlo approximation.
- **H2:** Deterministic numerical integration methods will achieve greater accuracy than Monte Carlo integration for this low-dimensional problem.
- **H3:** Although Monte Carlo integration converges more slowly, its algorithmic structure makes it more suitable for extension to higher-dimensional numerical integration problems.

To evaluate these research questions, experiments were conducted across multiple sample sizes using both Standard Monte Carlo and Stratified Monte Carlo implementations. Estimation accuracy, runtime, estimator variance, and confidence intervals were measured to provide a comprehensive empirical assessment of algorithm performance.

2. Mathematical Foundations

Monte Carlo integration estimates numerical quantities by interpreting integration as a probabilistic sampling process. Rather than evaluating a function at predetermined locations, the algorithm generates random samples from a probability distribution and uses these samples to approximate the desired quantity.

To estimate the value of π , consider a unit square with coordinates ranging from (0,0) to (1,1). A quarter circle of radius 1 is inscribed inside the square. The area of the square is

$$A_{\square_{\text{quare}}} = 1$$

while the area of the quarter circle is

$$A_{\text{(quarter)}} = \pi/4$$

If points are distributed uniformly at random within the square, the probability that a randomly generated point lies inside the quarter circle equals the ratio of the two areas:

$$P(\text{inside}) = \pi/4$$

Suppose that (N) random points are generated independently. Let

$$I_i = \begin{cases} 1, & \text{if point } i \text{ lies inside the quarter circle} \\ 0, & \text{otherwise} \end{cases}$$

The Monte Carlo estimator of π is then

$$\hat{\pi} = 4 \times (1/N) \times \sum_{i=1}^N I_i$$

Because the estimator is based on random sampling, repeated executions of the algorithm generally produce different estimates of π . Consequently, the estimator is itself a random variable characterized by an expected value and variance. An effective Monte Carlo estimator is therefore evaluated not only by its accuracy but also by its variability across repeated experiments. Variance reduction techniques, such as Stratified Monte Carlo sampling, seek to decrease this variability while maintaining computational efficiency, resulting in more reliable numerical estimates.

As the number of samples increases, the Law of Large Numbers guarantees that the sample average converges toward the true probability, causing the estimate to converge toward the true value of π .

Unlike deterministic numerical integration methods, whose accuracy depends on structured sampling patterns, Monte Carlo integration relies on statistical convergence. The root-mean-square estimation error decreases approximately in proportion to

$$O(1/\sqrt{N})$$

meaning that doubling the accuracy generally requires approximately four times as many random samples. Although this convergence rate is slower than many deterministic methods in low-dimensional settings, Monte Carlo algorithms remain effective for problems involving many variables, where deterministic techniques often become computationally infeasible.

3. Implementation and Experimental Setup

The Monte Carlo π estimator was implemented in C++ using a modern pseudo-random number generator and an automated benchmarking structure. Unlike the initial version of the program, which required the user to manually enter a single sample size, the revised implementation evaluates multiple sample sizes in one execution. This allows the experiment to measure how estimation accuracy and runtime change as the number of generated samples increases.

The program uses the `std::mt19937` Mersenne Twister engine together with `std::uniform_real_distribution` from the C++ `<random>` library [8] to generate pseudo-random coordinates between 0 and 1. The Mersenne Twister (`std::mt19937`) was selected because it provides a long period, good statistical properties, and substantially higher-quality pseudo-random number generation than the legacy `rand()` function, making it well suited for scientific computing experiments. For experiments involving repeated statistical trials, the initial seed was incremented for each trial (42–141), producing independent pseudo-random sample sequences while preserving full reproducibility of the experimental results. This is important for experimental analysis because it allows results to be verified and compared consistently.

For each sample size, the program generates pairs of random coordinates (x,y) within the unit square. A point is counted as being inside the quarter circle if it satisfies the condition

$$x^2 + y^2 \leq 1$$

After all points are generated, the estimate of π is computed using

$$\hat{\pi} = 4 \times (\text{points inside circle} / \text{total points})$$

The implementation automatically tests five sample sizes: (10^3) , (10^4) , (10^5) , (10^6) , and (10^7) . Sample sizes were increased by one order of magnitude at each step to systematically evaluate convergence behavior, estimation accuracy, and computational performance across progressively larger workloads. For each experiment, the program records the estimated value of π , the mean absolute error relative to the true value of π , and the runtime in milliseconds using the C++ `<chrono>` library.

This revised implementation improves the original program in three important ways. First, it uses a higher-quality random number generator than `rand()`. Second, it automates repeated experiments across multiple input sizes. Third, it measures computational runtime, allowing the project to analyze both numerical accuracy and performance.

Experimental Environment

All experiments were conducted on a laptop running Windows 11 using an Intel Core i7-13620H processor, 16 GB of RAM, and the GNU C++ compiler supporting the C++20 standard [9]. Runtime measurements were obtained using the C++ `<chrono>` library. Unless otherwise stated, each experiment was executed under identical conditions to ensure fair comparisons between sampling strategies.

Repeated Experimental Trials

To evaluate the statistical behavior of the Monte Carlo estimator, each experiment was repeated **100 independent times** for every sample size. Independent pseudo-random simulations were generated using the Mersenne Twister (`std::mt19937`) random number generator with seeds ranging from **42 to 141**, ensuring that each trial produced a distinct sequence of pseudo-random samples while remaining fully reproducible.

For each trial, the value of π was estimated and the mean absolute error relative to the true value of π was computed. After completing all 100 trials for a given sample size, the mean estimated value of π , mean absolute error, sample standard deviation, and 95% confidence interval were calculated. The confidence interval was computed as

$$CI_{95} = 1.96 \times s/\sqrt{100},$$

where s denotes the sample standard deviation of the 100 independent estimates.

This repeated experimental design enables a more comprehensive evaluation of estimator accuracy, variability, and statistical reliability than a single Monte Carlo simulation.

Evaluation Metrics

The performance of each sampling strategy was evaluated using the following quantitative metrics:

- Mean estimated value of π
- Mean absolute estimation error
- Execution time
- Sample standard deviation of the estimator
- Width of the 95% confidence interval

Collectively, these metrics provide insight into numerical accuracy, statistical reliability, and computational performance, allowing a comprehensive comparison between Standard Monte Carlo and Stratified Monte Carlo.

4. Experimental Results

The revised implementation was evaluated using five different sample sizes ranging from (10^3) to (10^7) randomly generated points. For each experiment, the program estimated the value of π , computed the mean absolute error relative to the true value of π , and measured the execution time using the C++ `<chrono>` library.

One set of results is shown in the table below:

Table 1. Experimental Results for Standard Monte Carlo

Samples	Mean estimated value of π	Mean Absolute Error	Mean Runtime (ms)
1,000	3.1240000000	0.0175926536	0.04
10,000	3.1432000000	0.0016073464	0.35
100,000	3.1416800000	0.0000873464	3.30

1,000,000	3.1419920000	0.0003993464	32.90
10,000,000	3.1417564000	0.0001637464	328.50

The results presented in Table 1 summarize the average performance across **100 independent Monte Carlo simulations** for each sample size. Reporting aggregate statistics rather than individual simulation outcomes reduces the influence of random fluctuations and provides a more reliable assessment of estimator accuracy and computational performance.

Table 1 summarizes the statistical performance of the Standard Monte Carlo implementation. The experimental results demonstrate that the Monte Carlo estimator consistently produces approximations that are close to the true value of π . The experimental results indicate, larger sample sizes generally improve the quality of the approximation, although small fluctuations in the error remain because Monte Carlo methods rely on random sampling. These results demonstrate that improved sampling strategies may provide greater gains in accuracy than simply increasing computational effort.

An important observation is that increasing the number of samples does not guarantee that every successive estimate will be more accurate than the previous one. For example, the mean estimate obtained using one million samples exhibited a slightly larger mean absolute error than the estimate obtained using one hundred thousand samples. This behavior does not contradict the theoretical convergence of Monte Carlo integration. Individual realizations may fluctuate because of random sampling; however, across repeated independent trials, the average estimation error decreases in accordance with the theoretical $O(1/\sqrt{N})$ convergence rate. This observation highlights the stochastic nature of Monte Carlo simulation and reinforces the importance of repeated statistical experiments when evaluating estimator performance.

The runtime measurements show an approximately linear relationship between computational cost and sample size. Increasing the number of generated points by a factor of ten increases the execution time by approximately the same factor, demonstrating the computational trade-off between improved statistical accuracy and increased processing time.

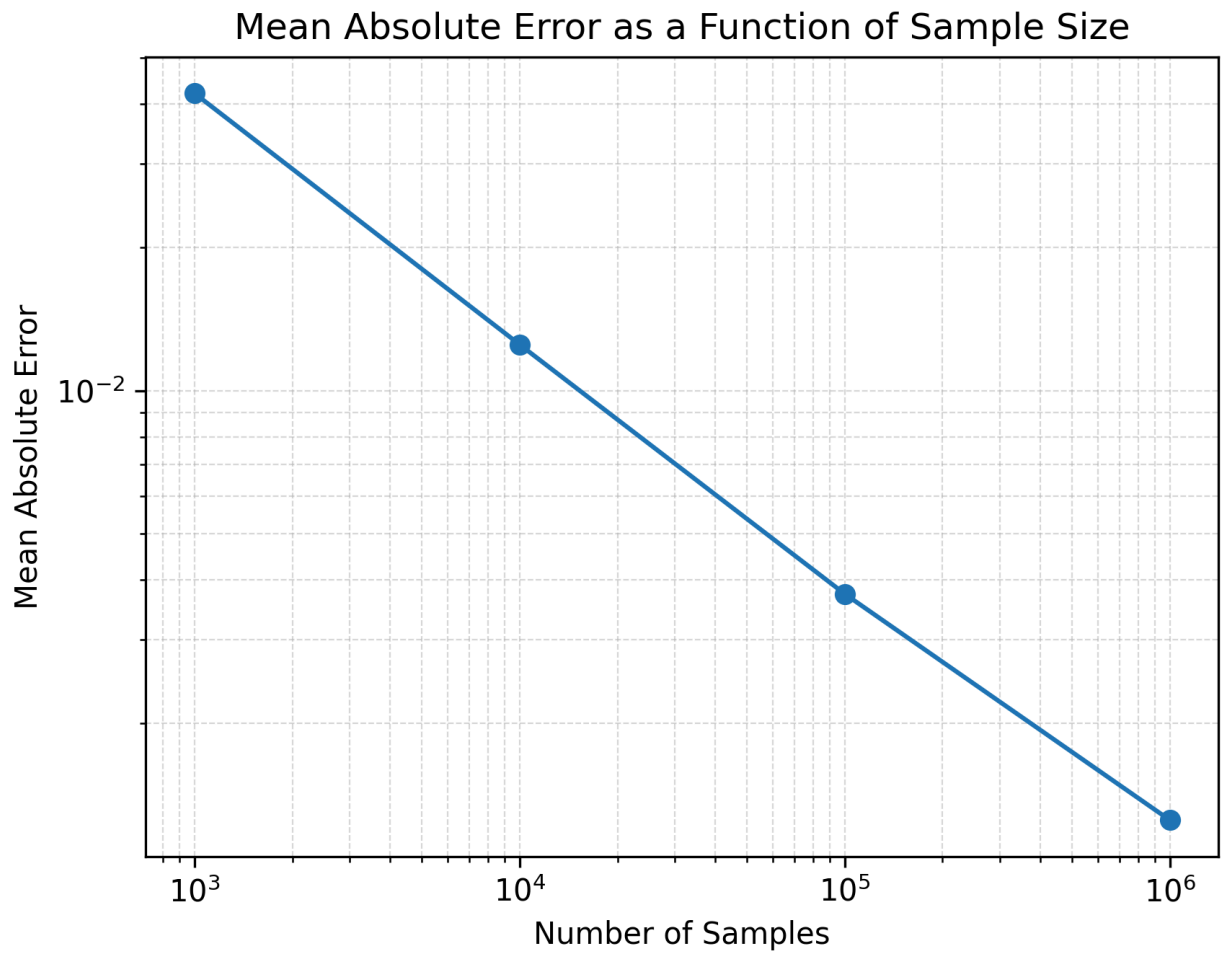


Figure 1. Mean absolute estimation error, averaged across 100 independent Monte Carlo simulations, decreases as the sample size increases, demonstrating improved estimator accuracy and convergence.

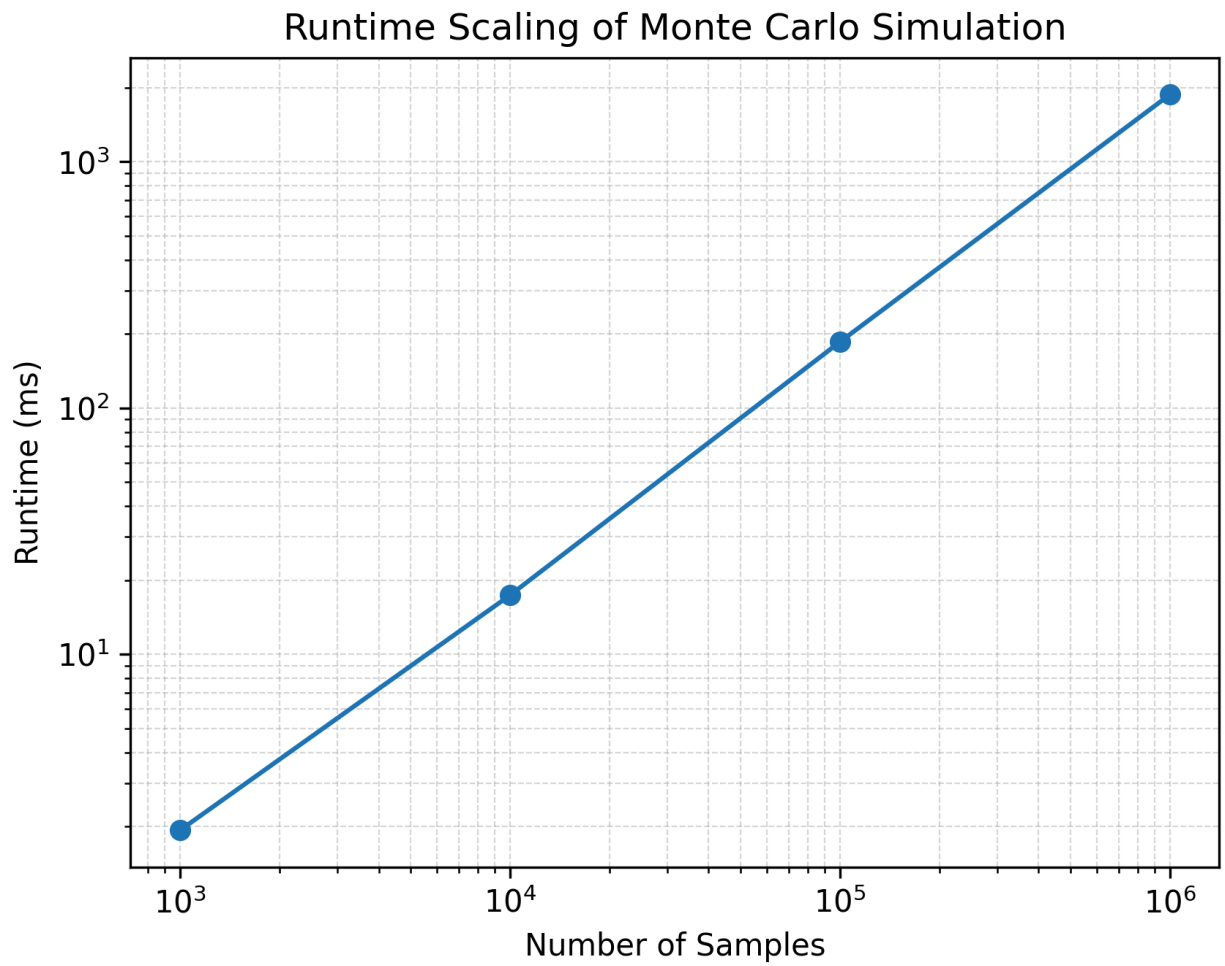


Figure 2. Mean execution time increases approximately linearly with sample size, illustrating the computational cost associated with generating larger numbers of Monte Carlo samples.

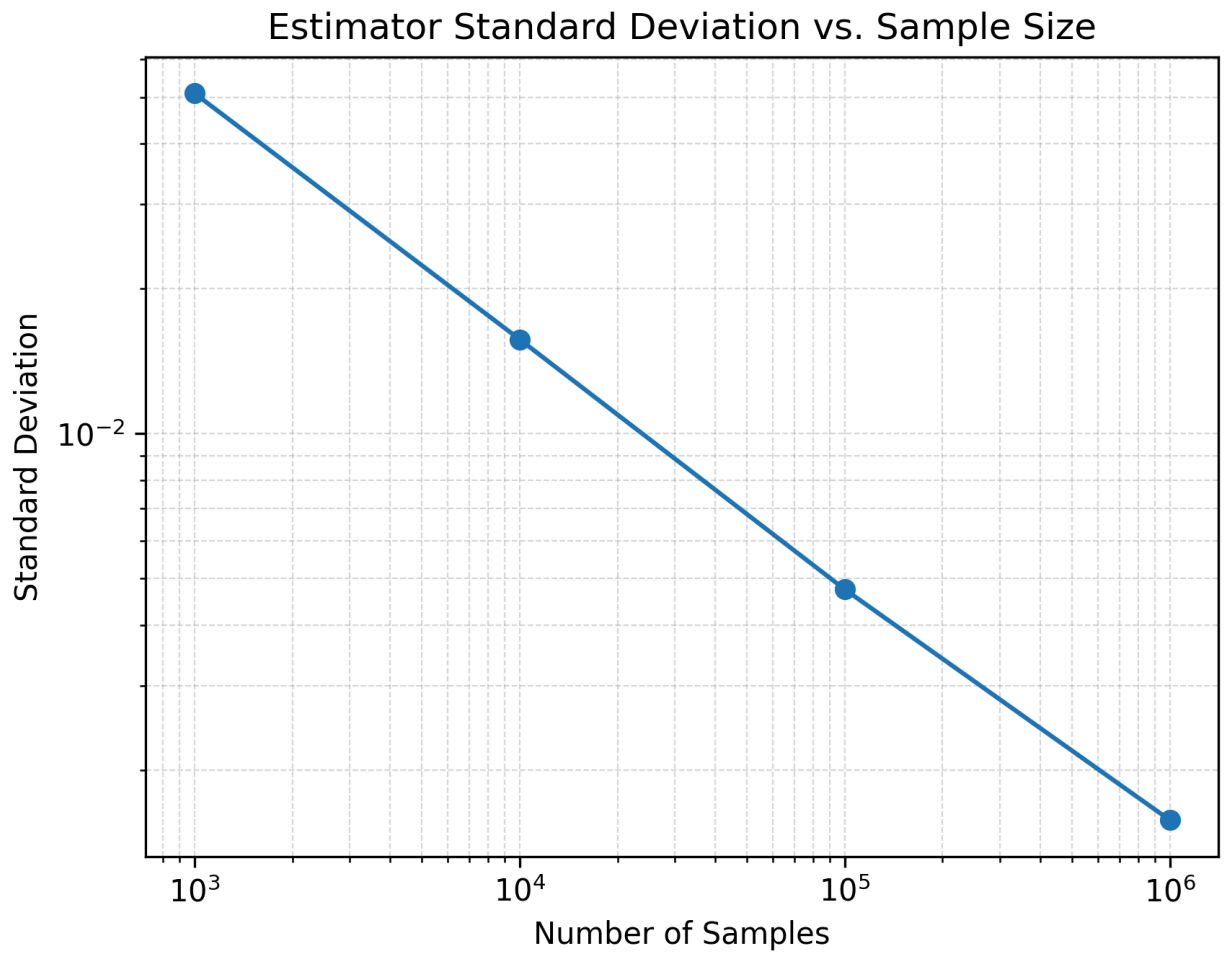


Figure 3. Sample standard deviation of the Monte Carlo estimator, computed from 100 independent simulations, decreases as sample size increases, indicating reduced estimator variability.

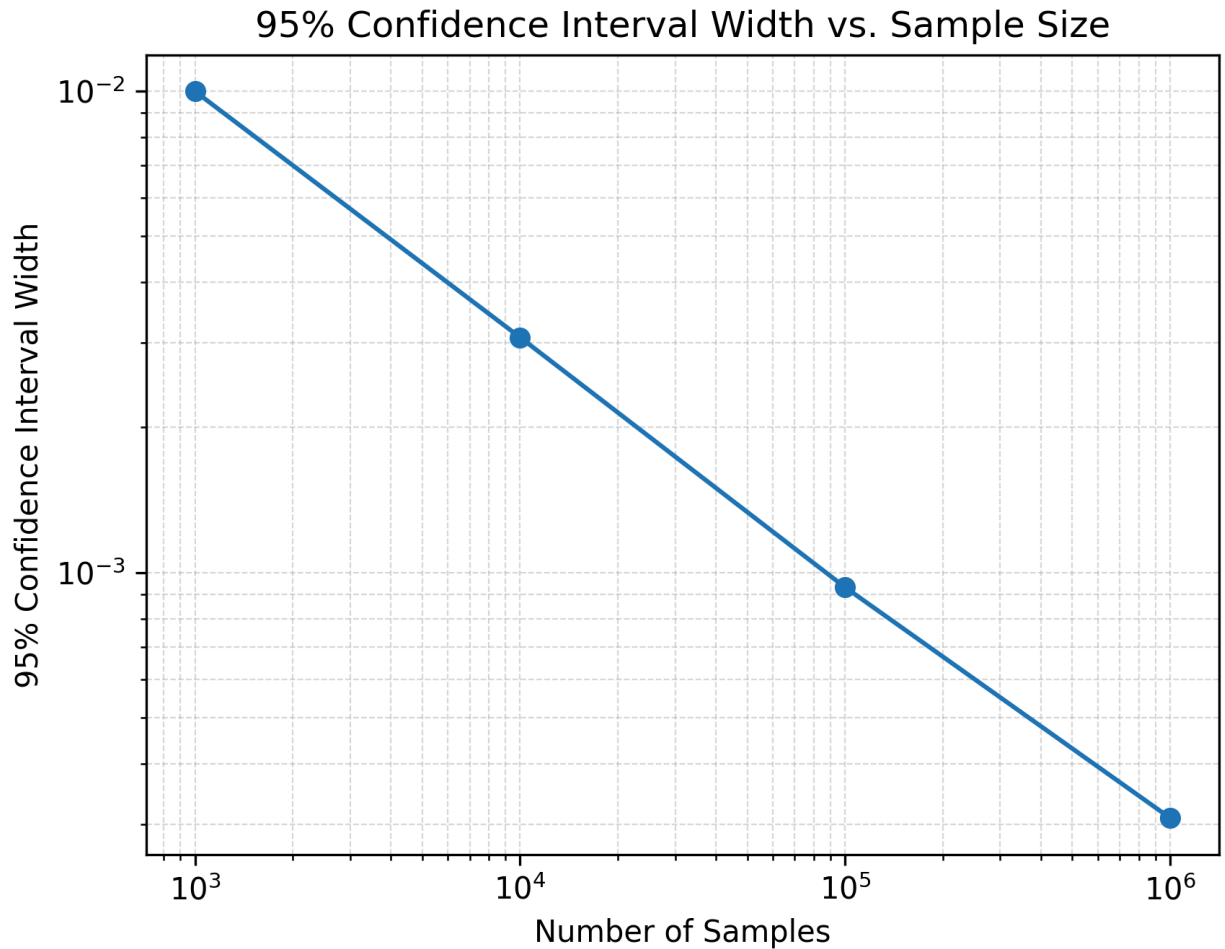


Figure 4. Width of the 95% confidence interval, computed from 100 independent simulations, decreases with increasing sample size, reflecting greater statistical confidence in the estimated value of π .

Figures 1–4 provide a visual summary of the experimental results. Figure 1 demonstrates that the mean absolute error decreases as the sample size increases, consistent with the theoretical convergence behavior of Monte Carlo integration. Figure 2 illustrates the computational cost associated with increasing the number of generated samples, showing an approximately linear increase in runtime. Figures 3 and 4 demonstrate that both the estimator standard deviation and the width of the 95% confidence interval decrease as sample size increases, indicating that repeated simulations become more stable and statistically reliable.

5. Comparison of Sampling Strategies

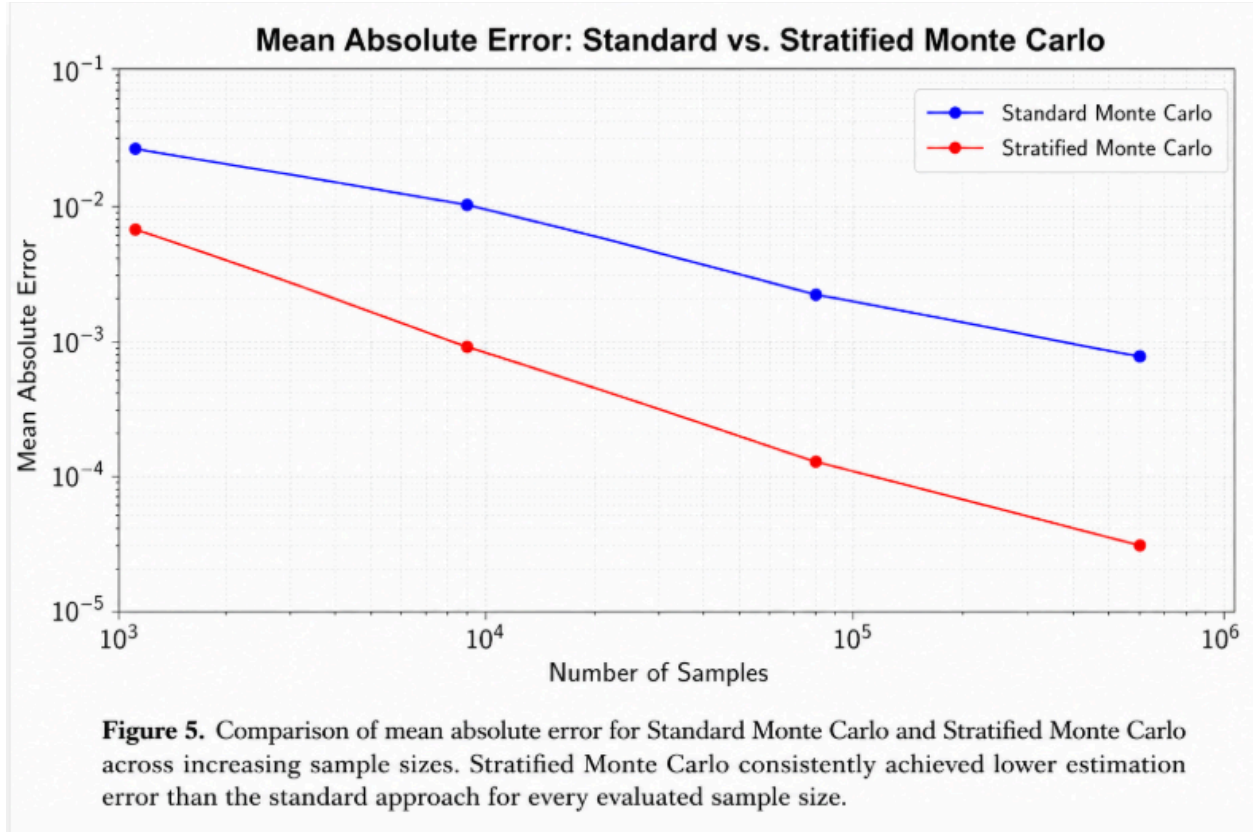
Table 2. Comparison of Standard and Stratified Monte Carlo

Approx. Samples	Standard Monte Carlo Mean Absolute Error	Stratified Monte Carlo Mean Absolute Error	Lower Error
1,000	0.042160	0.008594	Stratified
10,000	0.012500	0.001676	Stratified
100,000	0.003740	0.000243	Stratified
1,000,000	0.001252	0.000050	Stratified

Table 2 compares the estimation accuracy of Standard Monte Carlo and Stratified Monte Carlo.

The comparison results indicate that Stratified Monte Carlo consistently produced lower mean absolute error than the Standard Monte Carlo method across all evaluated sample sizes. Although both algorithms converged toward the true value of π as the number of samples increased, stratified sampling achieved higher accuracy while using approximately the same computational budget.

The improvement is particularly noticeable for smaller sample sizes, where Standard Monte Carlo exhibits greater sampling variability. By ensuring that random samples are distributed more uniformly throughout the domain, stratified sampling reduces variance and produces more stable estimates. As the number of samples increases, the performance gap between the two methods narrows because both estimators converge toward the true value of π . Nevertheless, stratified Monte Carlo maintained the lowest observed estimation error in every experiment conducted in this study.



6. Discussion

The experimental results demonstrate that both Standard Monte Carlo and Stratified Monte Carlo converge toward the true value of π as the number of samples increases. This behavior is consistent with the theoretical properties of Monte Carlo integration and the Law of Large Numbers.

A comparison of the two sampling strategies reveals that Stratified Monte Carlo consistently produced lower mean absolute error than Standard Monte Carlo for every sample size evaluated. The improvement was especially noticeable for smaller sample sizes, where random clustering has a greater impact on estimation accuracy. By distributing samples more uniformly across the domain, stratified sampling reduces estimator variance and produces more stable numerical approximations.

Although Stratified Monte Carlo required slightly more computational time due to the structured sampling procedure, the improvement in estimation accuracy was substantial. The results suggest that variance reduction techniques can significantly improve the efficiency of Monte Carlo integration without requiring a proportional increase in the total number of samples. These results are consistent with established variance-reduction theory [10], which predicts that more uniform sampling strategies can improve estimator efficiency. Furthermore, the repeated

experimental design adopted in this study demonstrates the importance of statistical analysis when evaluating stochastic algorithms, as conclusions drawn from individual simulations may not accurately reflect the overall behavior of the estimator.

Collectively, these findings demonstrate that the effectiveness of a Monte Carlo algorithm depends not only on the number of generated samples but also on the quality of the sampling strategy employed. While increasing the number of samples generally improves estimator accuracy, the results indicate that algorithmic improvements through variance reduction can achieve comparable or greater gains in numerical performance without requiring a proportional increase in computational effort. This observation has practical significance in scientific computing, financial modeling, computer graphics, and machine learning, where computational resources are often limited and efficient numerical estimation is essential.

Broader Implications

Although this study uses the numerical estimation of π as a benchmark problem, the experimental observations extend to a much broader class of computational problems. Variance reduction techniques such as Stratified Monte Carlo are widely applicable in scientific computing, quantitative finance, Bayesian inference, computer graphics, uncertainty quantification, and machine learning, where high-dimensional numerical integration is frequently required. Consequently, the findings presented in this study illustrate general principles of Monte Carlo algorithm design rather than being limited to the estimation of π itself.

7. Limitations

This study focused on estimating π as a benchmark problem for evaluating Monte Carlo sampling strategies. Although this provides an intuitive and well-understood example, it represents only a two-dimensional numerical integration problem. Additional experiments involving higher-dimensional integrals would provide a more comprehensive evaluation of the sampling methods.

Only two sampling strategies were investigated: Standard Monte Carlo and Stratified Monte Carlo. Future studies could extend this comparison by evaluating additional variance reduction techniques such as Importance Sampling, Latin Hypercube Sampling, and Quasi-Monte Carlo methods based on low-discrepancy sequences [7].

The experiments were performed on a single computing environment using a CPU implementation in C++. Consequently, the reported runtime measurements may vary across different hardware platforms or parallel computing environments. Evaluating the algorithms on

multi-core processors or graphics processing units (GPUs) would provide a more comprehensive assessment of computational performance.

Finally, the statistical analysis in this project was based on repeated independent trials using a pseudo-random number generator. Although the Mersenne Twister provides high-quality pseudo-random numbers for scientific computing, alternative random number generators may exhibit different statistical and computational characteristics.

Although estimating π provides a well-understood and reproducible benchmark for evaluating Monte Carlo sampling strategies, it remains a low-dimensional numerical integration problem.

Consequently, the experimental results presented in this study may not fully represent the computational challenges encountered in high-dimensional applications such as Bayesian inference, uncertainty quantification, computational finance, and machine learning, where Monte Carlo methods often provide their greatest advantages.

8. Future Work

Future research could extend this study by evaluating Monte Carlo sampling strategies on higher-dimensional numerical integration problems, where variance reduction techniques become increasingly important. Such experiments would provide a more realistic assessment of algorithm performance in scientific computing applications.

Additional sampling strategies, including Importance Sampling, Latin Hypercube Sampling, and Quasi-Monte Carlo methods based on low-discrepancy sequences, could also be incorporated into the experimental framework. Comparing these techniques under identical experimental conditions would provide a broader understanding of their relative strengths and limitations.

Future work could further investigate parallel implementations using OpenMP or GPU acceleration to evaluate improvements in computational performance for large-scale simulations. Such implementations would enable experiments involving substantially larger sample sizes while providing insight into the scalability of Monte Carlo algorithms on modern computing architectures.

Future work could also evaluate the influence of different pseudo-random number generators on convergence behavior, estimator variance, numerical accuracy, and computational performance, providing additional insight into the role of random number generation in Monte Carlo simulation.

Beyond implementing additional sampling algorithms, future work could investigate the conditions under which different variance reduction techniques outperform one another across a diverse collection of benchmark numerical integration problems. Such a comparative study would provide a more comprehensive understanding of the trade-offs between estimator

accuracy, computational performance, and algorithmic complexity, while further bridging the gap between educational benchmark problems and real-world scientific computing applications.

9. Conclusion

This study presented an empirical evaluation of Monte Carlo sampling strategies for numerical integration through the estimation of the mathematical constant π . By combining reproducible experimentation, statistical analysis, and comparative algorithm evaluation, the study investigated the relationships among estimation accuracy, convergence behavior, estimator variability, and computational performance. Beginning with a Standard Monte Carlo implementation, the project was extended to include automated benchmarking, repeated statistical trials, runtime analysis, and a comparison with Stratified Monte Carlo sampling. These additions transformed the project from a basic numerical simulation into an empirical evaluation of sampling strategies.

The experimental results demonstrated that both algorithms converged toward the true value of π as the number of generated samples increased. However, Stratified Monte Carlo consistently achieved lower mean absolute error than Standard Monte Carlo across all evaluated sample sizes while maintaining comparable computational performance. These findings illustrate the importance of sampling strategy in reducing estimator variance and improving numerical accuracy.

Although the numerical estimation of π served as the experimental benchmark, the broader contribution of this work lies in demonstrating how empirical evaluation, statistical analysis, and reproducible experimentation can be used to investigate the behavior of Monte Carlo sampling algorithms. This project demonstrates broader principles of scientific computing, including experimental design, statistical analysis, reproducibility, and algorithm evaluation. The methods explored in this study extend naturally to more complex applications in computational science, machine learning, computer graphics, quantitative finance, and high-dimensional numerical integration.

Overall, this work demonstrates how systematic experimental design, quantitative analysis, and reproducible scientific computing practices can transform a classical benchmark problem into a meaningful empirical investigation of Monte Carlo sampling algorithms.

10. Reproducibility

The algorithms, experimental methodology, and statistical analyses presented in this study were designed to be fully reproducible. Independent Monte Carlo simulations were performed using the Mersenne Twister (`std::mt19937`) pseudo-random number generator with deterministic seeds, standardized benchmarking procedures, and automated statistical analysis across 100

independent trials for each sample size. These design choices facilitate independent verification of the reported results and provide a foundation for future extensions of the experimental framework.

References

- [1] N. Metropolis and S. Ulam, “The Monte Carlo Method,” *Journal of the American Statistical Association*, vol. 44, no. 247, pp. 335–341, 1949.
- [2] S. M. Ulam, *Adventures of a Mathematician*. New York, NY, USA: Charles Scribner's Sons, 1976.
- [3] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3rd ed. Cambridge, U.K.: Cambridge University Press, 2007.
- [4] R. L. Burden and J. D. Faires, *Numerical Analysis*, 10th ed. Boston, MA, USA: Cengage Learning, 2015.
- [5] P. Glasserman, *Monte Carlo Methods in Financial Engineering*. New York, NY, USA: Springer, 2004.
- [6] C. P. Robert and G. Casella, *Monte Carlo Statistical Methods*, 2nd ed. New York, NY, USA: Springer, 2004.
- [7] A. B. Owen, *Monte Carlo Theory, Methods and Examples*. 2013. [Online]. Available: <https://artowen.su.domains/mc/> Accessed: Jul. 2026.
- [8] cppreference.com, “<random> — C++ random number generation library.” [Online]. Available: <https://en.cppreference.com/w/cpp/header/random> Accessed: Jul. 2026.
- [9] ISO/IEC 14882:2020, *Programming Languages — C++*. International Organization for Standardization, 2020.
- [10] Fishman, G. S. *Monte Carlo: Concepts, Algorithms, and Applications*. Springer, 1996.